

Práctica “Repo Auditable” — Arreglar versionado + registrar estados

Informe de auditoría de configuración · Gestión de la Configuración del Software (GCS)

- Asignatura** Gestión de la Configuración del Software
- Actividad** Semana 5 — Actividad 4 (3.1 Estados de configuración · 3.2 Gestión de versiones)
- Estudiante** ncuenca84 (ncuenca1984@gmail.com)
- Producto** API Inventario (mini) — GET /products , POST /products
- Repositorio** https://github.com/ncuenca84/repo_auditable
- Trazabilidad** Issue #1 (ISSUE-21) · Pull Request #2 (merge 1e6245e)
- Fecha** 2026-08-25

1. Resumen

Se recibió un repositorio con un *release* “medio fantasma” y trazabilidad incompleta. La auditoría detectó versionado no conforme a SemVer, tags con formatos mezclados, un **secreto versionado** (`config/.env` con `API_KEY`), un commit no técnico (“*update stuff*”) y cambios sin referencia a un Issue. El objetivo de la corrección fue restaurar la integridad y la trazabilidad del producto: normalizar el versionado a SemVer, eliminar el secreto del control de versiones, vincular todos los cambios a un Issue y un Pull Request, actualizar el `CHANGELOG` por versión y completar el registro de estados de configuración (*Status Accounting*). Resultado: repositorio limpio, auditable y con tags coherentes `v1.0.0` `v1.0.1` `v1.1.0` .

2. Hallazgos (≥ 8)

#	Hallazgo	Evidencia inicial	Severidad
H1	Tag <code>v1.0</code> no cumple SemVer (falta PATCH)	<code>git tag</code> → <code>v1.0</code>	Media
H2	Formatos de tag mezclados (<code>v1.0</code> vs <code>release-1.1</code>)	<code>git tag</code>	Media
H3	Secreto versionado: <code>config/.env</code> con <code>API_KEY=123456</code>	commit <code>206dde1</code>	Alta
H4	Commit no técnico “ <i>update stuff</i> ”	commit <code>206dde1</code>	Media
H5	<code>feat: add filter date</code> (no docs) sin documentación/estado ni Issue	commit <code>7556263</code>	Media
H6	<code>fix: hotfix pagos 1%</code> sin referencia a Issue	commit <code>64aalf6</code>	Baja
H7	<code>CHANGELOG</code> sin entradas por versión con SemVer	<code>CHANGELOG.md</code> inicial	Baja
H8	Sin registro de estados de configuración completo	<code>CM_STATUS_REGISTER.md</code> inicial	Media
H9	Falta <code>.gitignore</code> y plantilla de PR (evidencia no forzada)	árbol del repo	Baja

Captura — historial y tags ANTES de corregir

```
$ git log --oneline --decorate --graph
* 206dde1 (HEAD -> main) update stuff
* 7556263 (tag: release-1.1) feat: add filter date (no docs)
* 64aalf6 fix: hotfix pagos 1%
* 9872cb8 (tag: v1.0) chore: init baseline structure (ISSUE-20)

$ git tag
release-1.1      # formato inconsistente
v1.0             # no es SemVer (debe ser v1.0.0)
```

```
$ git status  
# config/.env versionado (secreto API_KEY)
```

Captura 1: estado inicial problemático (git log / tag / status).

3. Correcciones aplicadas

Hallazgo	Corrección	Evidencia
H1,H2	Eliminados tags inconsistentes; creados tags SemVer coherentes	v1.0.0 / v1.0.1 / v1.1.0
H3	<code>git rm --cached config/.env + .env.example + .gitignore</code>	commit 90ebfcf
H4	Commit “update stuff” corregido con commits técnicos trazables	PR #2
H5	REQ-003 documentado con criterios en SRS; baseline formal	commit 0cab1de
H6	Trazabilidad restaurada (Refs #1 en commits/PR)	Issue #1 / PR #2
H7	CHANGELOG reorganizado por versión (SemVer, 2026)	CHANGELOG.md
H8	Registro de estados completado (10 EC)	CM_STATUS_REGISTER.md
H9	Añadidos <code>.gitignore</code> y <code>.github/pull_request_template.md</code>	PR #2

Captura — historial, tags y estado DESPUÉS de corregir

```
$ git log --oneline --decorate --graph
* 9a56fad (HEAD, tag: v1.1.0) docs: add post-audit evidence (ISSUE-21)
* 1e6245e Merge PR #2: auditoria de versionado... (ISSUE-21)
|\
| * 5ede556 docs: update CHANGELOG (SemVer), status register, PR template (ISSUE-21)
| * 0cab1de docs: document REQ-003 filter, define baseline (ISSUE-21)
| * 90ebfcf chore: remove secret from repo, add env example + gitignore (ISSUE-21)
|/
* 206ddel update stuff
* 7556263 feat: add filter date (no docs)
* 64aalf6 (tag: v1.0.1) fix: hotfix pagos 1%
* 9872cb8 (tag: v1.0.0) chore: init baseline structure (ISSUE-20)

$ git tag -nl
v1.0.0 Baseline aprobada (estructura + SRS v1 + codigo + prueba)
v1.0.1 Hotfix (patch retrocompatible, sin nuevas funciones)
v1.1.0 REQ-003 filtro por fecha + correcciones de auditoria (ISSUE-21)

$ git status
nothing to commit, working tree clean # repo integro
$ git ls-files config/
config/.env.example # el secreto ya NO se versiona
```

Captura 2: estado final corregido (git log / tag / status limpio).

4. Versionado final (SemVer)

Tag	Commit	Tipo	Justificación SemVer
v1.0.0	9872cb8	Baseline	Primera línea base aprobada (estructura + REQ-001/002 + código + prueba).
v1.0.1	64aalf6	PATCH	Hotfix retrocompatible, sin nuevas funciones (incremento de PATCH).
v1.1.0	9a56fad	MINOR	Nueva funcionalidad REQ-003 (filtro por fecha), retrocompatible (incremento de MINOR).

Nota de auditoría: el hotfix (v1.0.1) se ordena antes de la nueva funcionalidad (v1.1.0) para mantener la monotonía de SemVer: un PATCH no puede contener una funcionalidad que introduce el MINOR siguiente. Un cambio incompatible de contrato habría exigido v2.0.0.

5. Registro de estados (CM_STATUS_REGISTER.md)

EC-ID	Elemento	Tipo	Versión/Ref	Estado	Evidencia
EC-01	docs/SRS/SRS_v1.md	Doc	v1.1.0	Baselined	commit + tag
EC-02	src/app.py	Code	v1.0.0 (9872cb8)	Integrado	commit + tag
EC-03	tests/test_app.py	Test	v1.0.0	Verificado	pytest: 1 passed

EC-04	CHANGELOG.md	Doc	v1.1.0	Aprobado	commit + release
EC-05	.gitignore	Config	90ebfcf	Aprobado	commit (ISSUE-21)
EC-06	config/.env.example	Config	90ebfcf	Integrado	commit (ISSUE-21)
EC-07	.github/pull_request_template.md	Process	v1.1.0	Aprobado	PR #2
EC-08	README.md	Doc	v1.0.0	Baselined	tag + release
EC-09	docs/CM/BASELINE.md	Doc	v1.1.0 (0cab1de)	Aprobado	commit (ISSUE-21)
EC-10	config/.env (secreto)	Config	—	Retirado	git rm --cached (90ebfcf)

6. Conclusión

Contar con **estados de configuración + versiones coherentes** reduce riesgos concretos: (1) *Seguridad* — se elimina la exposición de secretos versionados; (2) *Integridad del release* — SemVer y tags/releases evitan el “release fantasma” y permiten reproducir cualquier versión entregada; (3) *Trazabilidad* — cada cambio se vincula a un Issue y un PR, de modo que se sabe *qué* cambió, *por qué* y *con qué evidencia*; (4) *Control de cambios* — el registro de estados (*Status Accounting*) da visibilidad del estado de cada elemento de configuración. En una auditoría real, si no hay evidencia, el cambio no existió: este repositorio ahora la tiene.

Anexo — pendiente manual: por política de red del entorno de ejecución, el `git push` de tags está bloqueado (los tags están creados localmente y correctos). Para publicarlos como Releases: GitHub → *Releases* → *Draft a new release* → crear tag `v1.0.0`, `v1.0.1` y `v1.1.0` sobre los commits indicados en la sección 4; o desde una máquina con credenciales completas: `git push origin v1.0.0 v1.0.1 v1.1.0`.